

Cloud-Native Online Bookstore Project Report

DSAA 4040 Project Team2: Zhiyuan WANG, Yukun LI

May 21, 2026

Abstract

This report presents a cloud-native online bookstore project developed for DSAA 4040. The system implements a React/Nginx frontend, FastAPI backend services, and PostgreSQL database storage. It supports Docker Compose deployment, Kubernetes deployment, split backend services, monitoring, HPA autoscaling, and an optional CloudNativePG-based PostgreSQL high-availability mode. The final evaluation focuses on system design, deployment robustness, functional correctness, database consistency, failover behavior, and autoscaling behavior.

Contents

1	Project Objective and Motivation	2
2	System Architecture and Overall Design	3
2.1	High-Level Architecture	3
2.2	Request Routing Architecture	4
2.3	Backend Split Architecture	4
2.4	Data Flow for User Operations	5
2.5	Data Flow for Monitoring	5
2.6	Distributed Database in HA Mode	6
2.7	Database Architecture	6
2.8	Kubernetes Architecture	7
3	Implementation and Deployment Details	8
3.1	Backend Implementation	8
3.1.1	Book and Search API	9
3.1.2	Cart API	9
3.1.3	Order API and Transactional Consistency	9
3.1.4	Admin API	10
3.1.5	Monitoring API	10
3.2	Frontend Implementation	10
3.3	Database Implementation	11
3.4	Containerization	12
3.5	Docker Compose Deployment	12
3.6	Kubernetes Deployment Implementation	13
3.7	Distributed HA Build Workflow	13
3.8	ConfigMap, Secret, and Environment Configuration	14
3.9	Service, NodePort, and Public Demo Exposure	15
3.10	HPA and Autoscaling Implementation	15

3.11	Monitoring Dashboard Implementation	15
4	Testing, Evaluation, and Experimental Results	16
4.1	Experimental Environment	16
4.2	Distributed Build Workflow Verification	16
4.3	Functional Correctness	17
4.4	Order Idempotency and Stock Consistency	17
4.5	Kubernetes Deployment Verification	18
4.6	Split Backend Deployment	19
4.7	CloudNativePG High Availability	19
4.8	PostgreSQL Failover Experiment	20
4.9	Monitoring and Metrics	20
4.10	HPA Autoscaling Experiment	21
4.11	Evaluation Summary	21
4.12	Limitations	22
5	Challenges Encountered and Solutions	23
5.1	Stale Images in Minikube	23
5.2	Metrics-Server and HPA Readiness	23
5.3	Split Backend Routing	23
5.4	Order Consistency Under Retry or Failure	23
5.5	CloudNativePG Local Storage Permission	24
5.6	Image Preflight for Recreated Minikube Profiles	24
6	Future Work	24
7	Conclusion	25
8	Appendix A: Demo and Verification Commands	25
8.1	Full Distributed HA Build	25
8.2	Runtime Status Checks	25
8.3	Image Preflight	25
8.4	Public Demo Exposure	26
8.5	Functional and Consistency Tests	26
8.6	HA and HPA Experiments	26

1 Project Objective and Motivation

This project implements a cloud-native online bookstore system. The application supports the core user flow of an online bookstore: browsing books, searching the catalog, adding books to a shopping cart, placing an order, and viewing order history. In addition, it includes a simple Admin Demo page for catalog and inventory management, and a lightweight Monitoring Dashboard for observing Kubernetes deployment, HPA, pod, database, and metrics status during the demo.

The motivation of this project is to connect application development with cloud-native system design. This project goes further than traditional projects by introducing Docker, Docker Compose, Kubernetes manifests, ConfigMaps, Secrets, persistent storage, Services, Ingress routing, readiness/liveness probes, Horizontal Pod Autoscaling, RBAC, and deployment automation scripts. Therefore, the project demonstrates both software engineering functionality and cloud-native operational concerns.

In addition to the default single-database deployment, this project also includes an optional high-availability database mode through `DB_MODE=ha`. In this mode, PostgreSQL is deployed as a primary/replica cluster managed by CloudNativePG, where the primary handles write requests and replicas receive updates through write-ahead log replication. This design introduces key distributed database concepts into the bookstore system, including leader-follower replication, automatic failover, stable service endpoints, and consistency protection for critical operations such as checkout and inventory updates. Although the HA mode is demonstrated in a Minikube-based course environment rather than a production cluster, it provides a practical example of how a cloud-native application can improve database availability and fault tolerance.

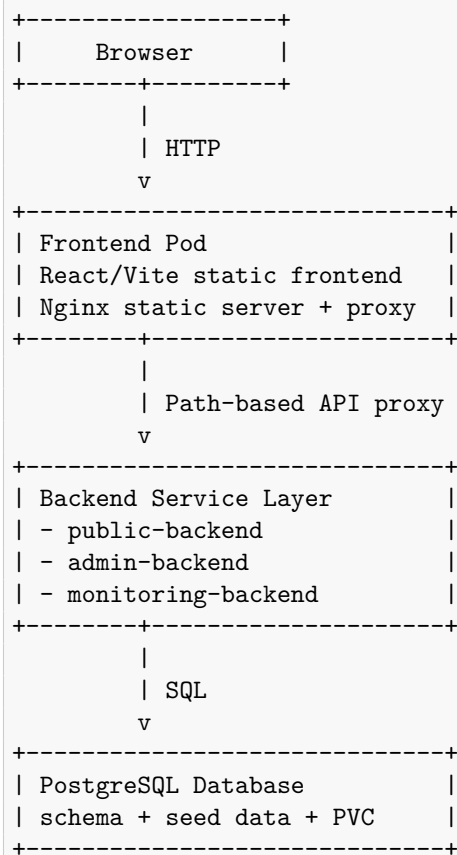
The final system is designed as a three-tier web application:

```
Browser
-> Frontend: React/Vite single-page application served by Nginx
-> Backend: FastAPI REST API services
-> Database: PostgreSQL persistent relational storage
```

2 System Architecture and Overall Design

2.1 High-Level Architecture

The project follows a three-tier architecture, but the Kubernetes version further refines the backend tier into three logical deployments. The high-level architecture is:



The frontend tier serves two roles. First, it provides the user interface through a React/Vite single-page application. Second, the frontend container uses Nginx to serve static files and proxy API requests to different backend services based on URL paths. This design lets the browser access the whole application through a single frontend entry point while the internal Kubernetes services remain separated.

The backend tier uses one shared FastAPI image, `bookstore-backend:latest`, but runs it in three Kubernetes deployments with different environment variables:

```
public-backend      -> public bookstore APIs
admin-backend       -> admin catalog and inventory APIs
monitoring-backend  -> Kubernetes cluster status API
```

This design has two advantages. First, it avoids building multiple backend images, which keeps the deployment workflow simple. Second, it allows different backend responsibilities to be isolated at the Kubernetes deployment level. The monitoring backend is the only backend deployment that needs Kubernetes API access, so only it receives a ServiceAccount and read-only RBAC permissions.

The database tier uses PostgreSQL. In the stable default workflow, it is deployed as a single PostgreSQL deployment with persistent storage through a PVC. The repository also includes an optional PostgreSQL HA experiment using CloudNativePG, but the stable default mode remains single-database mode for easier course demonstration.

2.2 Request Routing Architecture

The system uses path-based routing at both Nginx and Kubernetes Ingress levels.

For the NodePort/public demo path, the browser accesses the frontend service. Inside the frontend container, Nginx routes API requests as follows:

```
/api/admin/cluster/* -> monitoring-backend-service:8000
/api/admin/*         -> admin-backend-service:8000
/api/*              -> public-backend-service:8000
/                   -> React static frontend
```

The Kubernetes Ingress uses the same path split for the optional `bookstore.local` route:

```
bookstore.local/api/admin/cluster -> monitoring backend
bookstore.local/api/admin         -> admin backend
bookstore.local/api              -> public backend
bookstore.local/                 -> frontend
```

Using the same routing rules in Nginx and Ingress improves consistency. Whether accessing the application through the frontend NodePort or through the optional Ingress host, the same backend separation is preserved.

2.3 Backend Split Architecture

Although the backend code is stored in a single FastAPI application, `BACKEND_MODE` controls which route modules are registered when the application starts.

The supported modes are:

```
all      -> register all routes, mainly for local Docker Compose development
public   -> register public bookstore routes
admin    -> register admin book management routes
monitoring -> register cluster monitoring route
```

The route registration strategy is:

```
Health routes:      every mode
Admin routes:       all or admin
Public routes:      all or public
Monitoring routes:  all or monitoring
```

2.4 Data Flow for User Operations

A normal user operation follows this flow:

```
Browser UI action
-> frontend React event handler
-> frontend api.js fetch wrapper
-> Nginx /api proxy
-> public-backend FastAPI route
-> PostgreSQL query/transaction
-> JSON response
-> React state refresh and UI update
```

For example, when the user places an order:

1. The user clicks the “Place Order” button in the Cart component.
2. The frontend generates an Idempotency-Key.
3. api.js sends POST /api/orders with that idempotency key.
4. Nginx routes /api/orders to public-backend-service.
5. The public backend reads the cart, locks relevant book rows, validates stock, creates an order, creates order items, decrements stock, clears the cart, commits the transaction, and returns the created order.
6. The frontend refreshes books, cart, and order history.

This flow ensures that the user interface, backend business logic, and database state stay consistent after ordering.

2.5 Data Flow for Monitoring

The Monitoring Dashboard uses a separate flow:

```
React Monitoring Dashboard
-> GET /api/admin/cluster/status
-> Nginx routes to monitoring-backend-service
-> monitoring-backend loads Kubernetes config
-> Kubernetes Python client reads Deployment, HPA, Pods, metrics, DB status
-> aggregated JSON response
-> frontend dashboard tables and rolling charts
```

The frontend does not directly call the Kubernetes API. This is an important architectural decision because browsers should not receive Kubernetes credentials. Instead, the backend uses in-cluster ServiceAccount credentials and namespace-scoped read-only RBAC to safely retrieve only the required status information.

2.6 Distributed Database in HA Mode

Besides the default single-database deployment, this project also provides an optional high-availability database mode through `DB_MODE=ha`. In this mode, PostgreSQL is deployed as an operator-managed primary/replica cluster using CloudNativePG. The purpose of this design is to extend the bookstore from a basic cloud-native web application into a system that also demonstrates stateful distributed-system concepts. Unlike stateless frontend and backend Pods, which can be recreated or scaled horizontally without preserving local state, the database layer must guarantee durable storage, ordered replication, safe recovery, and protection against inconsistent writes. Therefore, the HA database mode focuses on how critical bookstore data, especially orders and inventory, can remain consistent when a database instance fails.

In the HA architecture, the current PostgreSQL primary works as the leader and handles all write operations through the stable `bookstore-postgres-rw` service. Replica instances act as followers and receive updates from the primary through PostgreSQL write-ahead log streaming. When the primary becomes unavailable, CloudNativePG monitors the cluster state and promotes a healthy replica to become the new primary. This maps directly to several important distributed-system concepts, including leader-follower replication, log replication, automatic failover, and service endpoint redirection. For the bookstore workload, the system chooses a CP/ACID-oriented design: checkout and inventory updates should be temporarily unavailable during uncertain failover periods rather than risking duplicate orders or oversold stock. To further handle retry ambiguity during partial failures, the order API supports an `Idempotency-Key`, ensuring that repeated checkout requests with the same key return the existing order instead of creating duplicate orders or decrementing stock multiple times.

Overall, the HA mode is not intended to claim production-grade distributed database infrastructure, because the experiment is still conducted in a Minikube-based course environment. However, it provides a clear and practical demonstration of how a cloud-native application can incorporate distributed database mechanisms, including primary/replica replication, failover recovery, stable write endpoints, and application-level consistency protection for critical business operations.

2.7 Database Architecture

The PostgreSQL database contains four main tables:

```
books
carts
orders
order_items
```

The `books` table stores the catalog and stock. The `carts` table stores the current cart for the demo user. The `orders` table stores order-level records, including total price, status, and idempotency key. The `order_items` table stores item-level order details and preserves the price at the time of purchase.

A simplified ER-style view is:

```
books 1 --- * carts
books 1 --- * order_items
orders 1 --- * order_items
```

Important database design decisions include:

- `books.price >= 0` and `books.stock >= 0` constraints prevent invalid catalog data.
- `carts.quantity > 0` prevents meaningless cart entries.
- `UNIQUE (user_id, book_id)` in `carts` ensures that a user cannot have duplicate cart rows for the same book.
- `orders.idempotency_key` with `UNIQUE (user_id, idempotency_key)` makes retry-safe order placement possible.
- `order_items.price` stores a purchase-time price snapshot, so historical orders remain correct even if catalog prices change later.
- `order_items.book_id` does not cascade delete books, which preserves historical order integrity.

2.8 Kubernetes Architecture

The Kubernetes architecture is implemented under the `k8s/` directory. The main resources include:

Resource	Purpose
Namespace <code>bookstore</code>	Isolates all project resources.
ConfigMap <code>bookstore-config</code>	Stores non-sensitive configuration such as DB host, DB name, and demo user ID.
Secret <code>bookstore-secret</code>	Stores demo database username and password.
PostgreSQL Deployment	Runs the database container.
PVC <code>postgres-pvc</code>	Persists PostgreSQL data across pod restarts.
PostgreSQL Service	Provides stable in-cluster DNS for the database.
PostgreSQL init Job	Applies schema and seed SQL.
Public backend Deployment	Runs public bookstore APIs with 2 replicas and HPA target.
Admin backend Deployment	Runs admin catalog/inventory APIs.
Monitoring backend Deployment	Runs monitoring API with read-only Kubernetes RBAC.
Backend Services	Provide stable internal access to backend deployments.
Frontend Deployment	Runs React/Nginx frontend with 2 replicas.
Frontend Service	Exposes frontend as NodePort 30080.
Ingress	Provides optional host/path routing through <code>bookstore.local</code> .
HPA	Scales <code>public-backend</code> from 2 to 5 replicas based on CPU.
RBAC	Gives monitoring backend read-only access to Kubernetes objects and pod metrics.

The public backend HPA targets CPU utilization. Because HPA utilization is calculated relative to CPU requests, the public backend deployment defines CPU requests and limits. The HPA uses

`minReplicas: 2`, `maxReplicas: 5`, and a target average CPU utilization of 50%.

In the default single-database mode, PostgreSQL is deployed as a normal Kubernetes Deployment with a PVC and a ClusterIP Service. In the distributed HA mode used for the final experiments, PostgreSQL is managed by CloudNativePG instead. The HA mode creates a `Cluster` resource named `bookstore-postgres`, three PostgreSQL instance pods, and stable services including `bookstore-postgres-rw`, `bookstore-postgres-ro`, and `bookstore-postgres-r`. The backend connects to `bookstore-postgres-rw`, so it does not need to know which PostgreSQL pod is currently the primary.

Mode	Database Deployment Design
<code>DB_MODE=single</code>	One PostgreSQL Deployment, one PVC, and one ClusterIP Service. This is the stable default workflow for basic Kubernetes deployment.
<code>DB_MODE=ha</code>	CloudNativePG manages three PostgreSQL instances with one primary and two replicas. The backend uses <code>bookstore-postgres-rw</code> as the stable write endpoint.

Table 2: Database deployment modes.

3 Implementation and Deployment Details

3.1 Backend Implementation

The backend is implemented with FastAPI. The application entry point creates a FastAPI app and conditionally includes route modules according to `BACKEND_MODE`. Health routes are always registered so that Kubernetes readiness/liveness probes and smoke tests work in all backend modes.

The backend route modules are organized by responsibility:

Module	Main Responsibility
<code>health.py</code>	Backend and database health checks.
<code>books.py</code>	Book listing and search.
<code>cart.py</code>	Cart retrieval, add, update, and delete operations.
<code>orders.py</code>	Transactional order placement and order history.
<code>admin_books.py</code>	Admin book creation, stock update, and deletion.
<code>cluster_status.py</code>	Kubernetes Deployment/HPA/Pod/metrics/database status aggregation.

Database access is centralized in `db.py`. The backend reads database configuration from environment variables such as `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD`. In the optional HA mode, `DB_WRITE_HOST` and `DB_READ_HOST` are also supported. Connections are created with `psycopg` and `dict_row`, allowing route handlers to return dictionary-like query results directly as JSON-compatible structures.

The backend uses Pydantic request schemas to validate inputs. For example, cart quantity must be positive, book price cannot be negative, and stock update payloads are controlled. This prevents part of the invalid input from reaching database logic.

3.1.1 Book and Search API

The book API supports listing books and keyword search. The frontend passes a search term through `/api/books?search=...`, and the backend queries the database for matching catalog records. This provides the basic catalog browsing function.

3.1.2 Cart API

The cart implementation supports four main operations:

```
GET    /api/cart
POST   /api/cart
PUT    /api/cart/{book_id}
DELETE /api/cart/{book_id}
```

When adding a book to the cart, the backend first verifies that the book exists. It then uses an upsert-style operation with `ON CONFLICT (user_id, book_id) DO UPDATE`, so adding the same book again increases the existing quantity instead of creating duplicate rows. The cart response includes each item and subtotal, plus a total cart price.

3.1.3 Order API and Transactional Consistency

The order placement implementation is the most important backend logic. It uses a database transaction and follows this sequence:

1. Ensure the idempotency schema exists for compatibility with existing demo databases.
2. If the client provides an idempotency key, check whether an order with the same key already exists for the demo user.
3. Read the current cart joined with book information.
4. Lock relevant book rows using `FOR UPDATE OF b`.
5. Reject empty carts.
6. Validate that each book has enough stock.
7. Calculate total order price.
8. Insert one row into `orders`.
9. Insert corresponding rows into `order_items`.
10. Decrement stock in `books`.
11. Delete the cart rows for the user.
12. Commit the transaction.

This implementation prevents several common consistency errors. The row lock prevents concurrent orders from both reading the same stock and causing negative inventory. The transaction prevents partial order creation: if any step fails before commit, the database state is rolled back. The idempotency key prevents duplicate orders when a client retries after timeout, failover, or ambiguous network failure.

3.1.4 Admin API

The Admin Demo API supports:

```
POST    /api/admin/books
PATCH  /api/admin/books/{book_id}/stock
DELETE  /api/admin/books/{book_id}
```

The stock update logic prevents stock from becoming negative. The delete logic checks whether the book is referenced by historical `order_items`. If so, it returns a conflict instead of deleting the book. This is important because order history should remain readable and meaningful after catalog changes.

3.1.5 Monitoring API

The monitoring backend provides:

```
GET /api/admin/cluster/status
```

It uses the Kubernetes Python client to read:

- public backend Deployment status;
- public backend HPA status;
- public backend Pods;
- pod CPU/memory metrics from `metrics.k8s.io`;
- PostgreSQL service and pod status;
- optional CloudNativePG cluster status in HA mode.

The API first tries to load in-cluster Kubernetes configuration. If that fails, it falls back to local kubeconfig, which makes local development easier. When the Metrics API is unavailable, it still returns a valid JSON response with `metricsAvailable: false` and warning messages instead of failing the whole dashboard.

3.2 Frontend Implementation

The frontend is implemented with React and Vite. The main application state is managed in `App.jsx`. It stores books, cart, orders, search text, backend health status, database health status, messages, errors, loading state, and current page.

The frontend uses three main views:

```
Store page
Admin Demo page
Monitoring Dashboard page
```

The main frontend modules are:

File/Component	Responsibility
<code>App.jsx</code>	Top-level state, initialization, page switching, event handlers.
<code>api.js</code>	Centralized fetch wrapper and API functions.
<code>BookList.jsx</code>	Book catalog display and search form.

File/Component	Responsibility
<code>Cart.jsx</code>	Cart item display, quantity update, removal, and order placement.
<code>OrderHistory.jsx</code>	Historical order display.
<code>AdminBooks.jsx</code>	Admin Demo catalog and stock operations.
<code>MonitoringDashboard.jsx</code>	Kubernetes status dashboard and rolling charts.
<code>StatusMessage.jsx</code>	User-facing success/error messages.

The `api.js` file centralizes HTTP requests. It uses `VITE_API_BASE_URL` when provided; otherwise, it uses relative paths. This design works well with Nginx proxying because the browser can call `/api/...` on the same origin as the frontend.

When placing an order, the frontend generates an idempotency key using `window.crypto.randomUUID()` when available, with a timestamp/random fallback. This key is sent through the `Idempotency-Key` header. This frontend-backend cooperation is important for safe retries and is also useful in the optional database HA/failover experiment.

The Monitoring Dashboard periodically calls the cluster status API and refreshes every few seconds. It stores chart history in browser memory.

3.3 Database Implementation

The database schema is implemented in `database/schema.sql`. It drops existing demo tables and recreates the four main tables. The seed file initializes demo catalog data for local and Kubernetes testing.

The schema includes several consistency constraints:

```
price >= 0
stock >= 0
cart quantity > 0
order total_price >= 0
order item quantity > 0
order item price >= 0
UNIQUE (user_id, book_id) for carts
UNIQUE (user_id, idempotency_key) for orders
```

These database-level constraints complement backend validation. Even if an invalid request bypassed frontend checks, the database would still reject invalid negative prices, negative stock, non-positive quantities, or duplicate idempotency keys.

The PostgreSQL init strategy differs slightly between Docker Compose and Kubernetes:

- In Docker Compose, `schema.sql` and `seed.sql` are mounted into `/docker-entrypoint-initdb.d/`, so PostgreSQL initializes automatically on first volume creation.
- In Kubernetes, a ConfigMap and init Job are used to apply schema and seed SQL after the PostgreSQL pod is ready.

3.4 Containerization

The backend Dockerfile uses `python:3.12-slim`, installs dependencies from `requirements.txt`, copies the `app/` directory, exposes port 8000, and runs Uvicorn:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

The frontend Dockerfile uses a multi-stage build:

1. Build stage:
 - uses `node:20-alpine`;
 - installs npm dependencies;
 - builds the Vite app.
2. Runtime stage:
 - uses `nginx:alpine`;
 - copies `nginx.conf`;
 - copies built static files to `/usr/share/nginx/html`;
 - exposes port 80.

This reduces the final frontend image size and separates frontend build dependencies from runtime serving.

3.5 Docker Compose Deployment

Docker Compose is used for local full-stack integration. It defines three services:

Service	Image/Build	Port	Purpose
db	postgres:16	5432:5432	PostgreSQL database with schema/seed initialization.
backend	bookstore-backend:latest, built from <code>./backend</code>	8000:8000	FastAPI backend.
frontend	bookstore-frontend:latest, built from <code>./frontend</code>	8080:80	React/Nginx frontend.

The backend depends on the database health check, so it starts only after PostgreSQL is ready. This reduces startup race conditions. The frontend depends on the backend and exposes the user interface at `http://localhost:8080`.

Helper scripts support Compose development:

```
./scripts/compose-up.sh
./scripts/compose-status.sh
./scripts/compose-logs.sh
./scripts/compose-down.sh
```

API smoke testing can be performed with:

```
./scripts/test-api.sh
./scripts/test-admin-api.sh
```

3.6 Kubernetes Deployment Implementation

The Kubernetes deployment is the main cloud-native part of the project. It uses Minikube as the target cluster. The recommended deployment script is:

```
./scripts/k8s-rebuild-and-deploy.sh
```

This script is more than a simple `kubectl apply`. It handles the local image lifecycle problem caused by stable local tags such as `bookstore-backend:latest` and `bookstore-frontend:latest`.

The workflow is:

```
build backend image
-> verify backend image contains required route files
-> build frontend image
-> save current application replica counts
-> scale backend/frontend deployments to zero
-> remove old same-tag images inside Minikube
-> load fresh images into Minikube
-> apply Kubernetes manifests
-> restore replica counts
-> wait for rollouts
-> verify backend pods contain expected route files
```

This design addresses a real deployment issue: in Minikube, using the same image tag repeatedly can cause stale images to remain in running pods. `kubectl apply` only updates Kubernetes resource definitions; it does not guarantee that a pod will use a newly built local image with the same tag. The script therefore explicitly removes old images, loads new images, and restarts pods.

The Kubernetes application layer consists of:

- `public-backend`: 2 replicas, HPA target, public bookstore APIs.
- `admin-backend`: 1 replica, admin catalog APIs.
- `monitoring-backend`: 1 replica, Kubernetes monitoring API, ServiceAccount and RBAC.
- `frontend`: 2 replicas, React/Nginx frontend.
- `postgres`: 1 replica, persistent PostgreSQL with PVC.

Readiness and liveness probes are configured for frontend and backend pods. The backend probes call `/api/health`; the frontend probes call `/`. These probes help Kubernetes determine when pods are ready to receive traffic and when they should be restarted.

3.7 Distributed HA Build Workflow

For the final distributed-system demo, the project provides a one-command HA workflow through `scripts/k8s-distributed-ha-rebuild-all.sh`. This script is used to build or refresh the multi-node Minikube deployment without manually running each Kubernetes stage.

The workflow performs the following steps:

1. start or reuse the `bookstore-distributed` Minikube profile;
2. run image preflight and load required images into the Minikube profile;
3. install or check the CloudNativePG operator;
4. prepare CloudNativePG local storage and static local PVs;
5. create the CloudNativePG PostgreSQL HA cluster;
6. initialize the database schema and seed data;
7. deploy the frontend and split backend services;
8. verify application rollout and NodePort availability;
9. collect distributed HA evidence;
10. optionally expose the frontend through a public demo port.

The main demo command is:

```
AUTO_FIX_CNPG_PVC_PERMISSIONS=1 \  
EXPOSE_DEMO=1 \  
PUBLIC_PORT=3000 \  
NODE_PORT=30080 \  
MINIKUBE_PROFILE=bookstore-distributed \  
NODES=3 \  
CPUS=2 \  
MEMORY=4096 \  
DB_MODE=ha \  
./scripts/k8s-distributed-ha-rebuild-all.sh
```

This workflow keeps the Minikube profile itself intact. For a clean demo rebuild, the application namespace and demo database storage can be reset without running `minikube delete`. This avoids repeatedly downloading the Minikube base image and reduces demo setup risk.

3.8 ConfigMap, Secret, and Environment Configuration

The project separates non-sensitive and sensitive configuration:

- **ConfigMap** stores:
 - database mode;
 - database host;
 - database port;
 - database name;
 - application demo user ID.
- **Secret** stores:
 - database username;
 - database password.

The backend deployments use `envFrom` to load both the ConfigMap and Secret into container environment variables. This keeps container images independent of environment-specific values. The current Secret is explicitly demo-only and should not be treated as production-grade secret management.

3.9 Service, NodePort, and Public Demo Exposure

Internal backend services are `ClusterIP` services, which are accessible inside Kubernetes but not directly exposed publicly. The frontend service is a `NodePort` service on port 30080.

For a cloud VM running Minikube, the Minikube IP is usually internal to the VM. Therefore, a browser outside the VM cannot directly access `$(minikube ip):30080`. The project provides an exposure script:

```
PUBLIC_PORT=3000 NODE_PORT=30080 ./scripts/k8s-expose-demo.sh
```

This script uses iptables to forward public TCP port 3000 on the VM to the Minikube NodePort. This is appropriate for a course demo, but in production a cloud LoadBalancer or public Ingress with DNS and TLS would be more standard.

3.10 HPA and Autoscaling Implementation

The HPA is configured to scale `public-backend`:

```
minReplicas: 2
maxReplicas: 5
target average CPU utilization: 50%
```

Only the public backend is autoscaled because it handles normal user traffic such as catalog browsing, cart operations, and order placement. Admin and monitoring workloads are low-frequency demo functions and do not need the same scaling behavior.

The HPA depends on `metrics-server`. The repository includes `k8s-fix-metrics-server.sh` because `metrics-server` may fail in Minikube due to image pull or tag/digest issues. The repair script enables the addon, detects failures, patches the image when needed, waits for rollout, verifies `kubectl top`, and checks that the HPA no longer reports unknown CPU metrics.

The HPA demo script uses `hey` to generate sustained HTTP load. It records baseline HPA and pod status, runs load against the bookstore API, monitors scaling behavior, prints load test output, and optionally waits for scale-down evidence.

3.11 Monitoring Dashboard Implementation

The Monitoring Dashboard is implemented as a lightweight in-app observability feature. It does not store metrics in a database. Instead, it periodically calls the monitoring backend and keeps rolling chart samples in browser memory.

The dashboard shows:

- deployment replica status;
- HPA replica and CPU status;
- public backend pod table;
- pod readiness and restart count;
- CPU and memory metrics if available;
- warnings when metrics are unavailable;
- database mode and database pod/service status.

In the final version, the Monitoring Dashboard refreshes every two seconds during the demo. The monitoring backend returns a server-side timestamp and no-cache headers so that the browser does not reuse stale monitoring responses. The endpoint also returns best-effort recent Kubernetes events for HPA, Deployment, and CloudNativePG resources. If events or metrics are temporarily unavailable, the endpoint still returns a JSON response with warning messages instead of failing the whole dashboard. This design makes the dashboard useful during HPA and failover demonstrations.

4 Testing, Evaluation, and Experimental Results

This section summarizes the experimental validation of the cloud-native online bookstore system. Since the final demo will show the user interface and runtime behavior directly, this report focuses on the tested system behavior and measured results instead of screenshots.

The evaluation covers five main aspects: functional correctness, Kubernetes deployment correctness, database consistency, PostgreSQL high availability, and horizontal autoscaling.

4.1 Experimental Environment

The final experiments were conducted on a three-node Minikube profile named `bookstore-distributed`. The Kubernetes namespace was `bookstore`. The application was deployed in `DB_MODE=ha`, using CloudNativePG to manage a three-instance PostgreSQL cluster.

Node	Role	Internal IP
<code>bookstore-distributed</code>	control-plane	192.168.49.2
<code>bookstore-distributed-m02</code>	worker	192.168.49.3
<code>bookstore-distributed-m03</code>	worker	192.168.49.4

Table 6: Minikube multi-node experimental environment.

The application was accessed through the frontend NodePort service at port 30080. The main evaluated endpoint was:

```
http://192.168.49.2:30080
```

4.2 Distributed Build Workflow Verification

Before running functional and HA experiments, the distributed deployment workflow was verified. The one-command script `k8s-distributed-ha-rebuild-all.sh` was used as the main build entry for the final HA environment. The script started or reused the `bookstore-distributed` Minikube profile, checked required images, prepared CloudNativePG local storage, created the HA database, initialized schema and seed data, deployed the application, verified rollout, and collected distributed HA evidence.

After the workflow completed, the frontend NodePort endpoint returned HTTP 200, the backend health endpoint returned OK, the database health endpoint reported a connected database, and the monitoring endpoint returned cluster status JSON. This confirms that the project can be rebuilt through a repeatable deployment workflow instead of relying on manual Kubernetes commands.

4.3 Functional Correctness

The functional tests verified the main bookstore workflow, including backend health, database connectivity, book listing, cart operations, order placement, and order history retrieval.

The API smoke test successfully completed the following user flow:

- checked backend health with `/api/health`;
- checked database connectivity with `/api/health/db`;
- listed seeded books with `/api/books`;
- retrieved and updated the shopping cart;
- placed an order;
- retrieved order history.

The database health endpoint returned that the database was connected, and the order placement endpoint successfully created an order with one order item. This confirms that the public backend, frontend routing, and PostgreSQL HA database were correctly integrated.

The admin API test also passed. It created a temporary book, increased its stock, decreased its stock, deleted the temporary book, and listed the catalog again. This verifies the correctness of the admin backend and the `/api/admin/books` route.

Test Item	Result
Backend health check	PASS
Database health check	PASS
Book listing	PASS
Cart operation	PASS
Order placement	PASS
Order history retrieval	PASS
Admin book creation	PASS
Admin stock update	PASS
Admin book deletion	PASS

Table 7: Functional correctness summary.

4.4 Order Idempotency and Stock Consistency

Order placement is a critical state-changing operation. The system uses an `Idempotency-Key` to avoid duplicate orders and duplicate stock deduction when the same request is retried.

The order consistency test submitted repeated order requests with the same idempotency key. The observed result was:

```
Stock before: 19
PASS: repeated Idempotency-Key returned order 2
Stock after: 18
PASS: stock decremented exactly once
```

Metric	Observed Result
Initial stock	19
Final stock	18
Repeated idempotency key	Returned the same order
Stock decrement count	Exactly once
Final result	PASS

Table 8: Order idempotency and stock consistency result.

This result shows that repeated submission of the same logical order does not create duplicate orders and does not reduce inventory more than once. This behavior is important for retry safety during network failures, client timeouts, or database failover.

4.5 Kubernetes Deployment Verification

The Kubernetes verification confirmed that all major application components were deployed and ready. The final deployment included:

- two `frontend` pods;
- two `public-backend` pods;
- one `admin-backend` pod;
- one `monitoring-backend` pod;
- three CloudNativePG PostgreSQL pods.

The rollout verification succeeded for all application deployments: `public-backend`, `admin-backend`, `monitoring-backend`, and `frontend`. The frontend service was exposed as a `NodePort` service on port 30080. The local Kubernetes service test confirmed that:

- the `bookstore` namespace existed;
- all pods were running or completed;
- the frontend `NodePort` returned HTTP 200;
- `/api/health` reported backend OK;
- `/api/health/db` reported database OK;
- `/api/books` returned book data;
- `/api/admin/cluster/status` returned cluster status JSON.

The deployment script also verified that the running backend pods contained the expected route file `admin_books.py`. This was used to confirm that the running pods used the updated backend image rather than a stale local image.

Kubernetes Check	Result
Application rollout	PASS
Frontend NodePort service	PASS
Backend health route	PASS
Database health route	PASS
Books API route	PASS
Monitoring API route	PASS
Running image freshness check	PASS

Table 9: Kubernetes deployment verification summary.

4.6 Split Backend Deployment

The Kubernetes deployment separates backend responsibilities into three services. The **public-backend** handles user-facing bookstore operations, the **admin-backend** handles catalog management, and the **monitoring-backend** handles cluster status reporting.

Deployment	Replicas	Service	Responsibility
public-backend	2	public-backend-service	Public APIs
admin-backend	1	admin-backend-service	Admin APIs
monitoring-backend	1	monitoring-backend-service	Cluster status API
frontend	2	frontend-service	React/Nginx frontend

Table 10: Split backend deployment summary.

The endpoint information showed that the public backend had two service endpoints, while the admin and monitoring backends each had one endpoint. This confirms that the system was not deployed as a single monolithic backend pod, but as separated Kubernetes backend deployments.

4.7 CloudNativePG High Availability

The database was deployed using CloudNativePG in HA mode. The final cluster contained three PostgreSQL instances. At the time of evaluation, the cluster reported three ready instances out of three, and its status was healthy.

PostgreSQL Pod	Role	Kubernetes Node
bookstore-postgres-1	replica	bookstore-distributed-m03
bookstore-postgres-2	primary	bookstore-distributed-m02
bookstore-postgres-3	replica	bookstore-distributed

Table 11: CloudNativePG primary and replica placement.

The HA service layout was also verified:

- **bookstore-postgres-rw** selected the current primary;
- **bookstore-postgres-ro** selected the two replicas;

- `bookstore-postgres-r` selected all PostgreSQL instances.

The distributed HA evidence script reported:

```
postgresDistinctNodes=3
publicBackendDistinctNodes=2
STRONG PASS: PostgreSQL HA pods use 3 distinct Kubernetes nodes.
```

This verifies that the PostgreSQL HA pods were spread across all three Minikube nodes. Although this is still a Minikube simulation on a single VM, it demonstrates Kubernetes-level multi-node placement and database role separation.

4.8 PostgreSQL Failover Experiment

The failover experiment tested whether the system could recover when the current PostgreSQL primary pod was deleted. The script first identified the current primary, inserted a pre-failover marker through the read-write service, deleted only the primary pod, waited for a new primary, and then checked whether data remained available and new writes could still be performed.

The observed failover result is summarized in Table 12.

Metric	Observed Result
Old primary	<code>bookstore-postgres-1</code>
Old primary node	<code>bookstore-distributed-m03</code>
New primary	<code>bookstore-postgres-2</code>
New primary node	<code>bookstore-distributed-m02</code>
Failover duration	19 seconds
Pre-failover data preserved	Yes
Post-failover write succeeded	Yes
Primary moved to a different node	Yes
Final result	PASS

Table 12: PostgreSQL failover experiment result.

The result shows that CloudNativePG successfully promoted a replica on a different Kubernetes node. The `bookstore-postgres-rw` service recovered and continued to route writes to the new primary. The pre-failover marker was still readable after failover, and a post-failover marker was inserted successfully.

4.9 Monitoring and Metrics

The monitoring endpoint `/api/admin/cluster/status` returned deployment, HPA, pod, and database HA information. In the HA deployment, it reported:

- database mode: `ha`;
- write host: `bookstore-postgres-rw`;
- read host: `bookstore-postgres-ro`;
- primary pod: `bookstore-postgres-2`;

- two replica pods;
- three ready PostgreSQL instances.

Initially, HPA CPU metrics were unavailable because the Metrics API was not ready. The metrics-server repair script enabled and repaired metrics-server, after which node and pod metrics became available. The HPA then reported CPU utilization correctly:

```
public-backend-hpa    cpu: 2%/50%    MINPODS=2    MAXPODS=5    REPLICAS=2
```

This confirms that the monitoring endpoint and Kubernetes metrics pipeline were both functional after repair.

4.10 HPA Autoscaling Experiment

The HPA experiment evaluated whether the `public-backend` deployment could scale out under sustained load. Before the load test, the deployment was running at the HPA minimum of two replicas. During the load test, CPU utilization increased above the target threshold.

The observed result is summarized below:

Metric	Observed Result
Initial public-backend replicas	2
HPA CPU target	50%
Observed CPU utilization under load	454%
Intermediate scale-out	2 to 4 replicas
Maximum observed replicas	5
Final result	PASS

Table 13: HPA autoscaling experiment result.

The deployment scaled from two replicas to four replicas and then to five replicas. This confirms that HPA reacted to sustained CPU load and increased the number of backend pods.

4.11 Evaluation Summary

Table 14 summarizes the final evaluation results.

Category	Evidence	Result
Frontend availability	NodePort returned HTTP 200.	PASS
Backend health	/api/health returned OK.	PASS
Database health	/api/health/db returned connected.	PASS
Catalog API	/api/books returned seeded book data.	PASS
Cart and order flow	API smoke test completed successfully.	PASS
Admin flow	Temporary book create/update/delete succeeded.	PASS
Order consistency	Repeated idempotency key decremented stock once.	PASS
Kubernetes deployment	All application deployments rolled out.	PASS
Split backend	Public, admin, and monitoring services were deployed separately.	PASS
PostgreSQL HA	CloudNativePG cluster had 3/3 ready instances.	PASS
Node distribution	PostgreSQL pods used 3 distinct Kubernetes nodes.	PASS
Failover	Primary moved to another node in 19 seconds.	PASS
Data preservation	Pre-failover marker remained after failover.	PASS
Post-failover write	New write succeeded through <code>bookstore-postgres-rw</code> .	PASS
Monitoring	Cluster status endpoint returned HA and deployment status.	PASS
Metrics	metrics-server repair enabled HPA CPU metrics.	PASS
Autoscaling	public-backend scaled from 2 to 5 replicas.	PASS

Table 14: Final evaluation summary.

4.12 Limitations

The experiments confirm the main project goals, but several limitations remain.

First, the distributed HA experiment uses Minikube multi-node mode on a single VM. It demonstrates Kubernetes-level node scheduling and failover behavior, but it does not provide real physical fault isolation. A production deployment should use a real multi-node Kubernetes cluster or managed Kubernetes service.

Second, the admin interface is intentionally unauthenticated for course demonstration. It should not be considered production-ready.

Third, the application currently connects to the read-write PostgreSQL service for all database queries. The read-only replica service exists, but application-level read splitting is left as future work.

Finally, the HPA experiment depends on metrics-server availability and enough sustained CPU load. In the final experiment, metrics-server was repaired successfully and HPA scale-out was observed, but this dependency should be considered when reproducing the experiment.

Overall, the evaluation shows that the system provides a working cloud-native bookstore application with Kubernetes deployment, split backend services, transactional order placement, idempotency protection, PostgreSQL HA, primary failover, monitoring support, and HPA autoscaling.

5 Challenges Encountered and Solutions

5.1 Stale Images in Minikube

A major challenge is that the project uses stable local image tags such as `bookstore-backend:latest` and `bookstore-frontend:latest`. In Minikube, if a pod already has an image with the same tag, simply running `kubectl apply` may not update the running code. This can cause confusing errors such as a new route returning 404 even though the source code has already been modified.

The solution is implemented in `k8s-rebuild-and-deploy.sh`. The script rebuilds images, removes old same-tag images inside Minikube, loads fresh images, restarts relevant pods, and verifies that the running pod contains expected files. This makes the deployment process repeatable and reduces manual debugging.

5.2 Metrics-Server and HPA Readiness

HPA requires the Kubernetes Metrics API. In Minikube, metrics-server may fail due to image pull problems or invalid tag/digest combinations. When this happens, `kubectl top` fails and HPA may show `<unknown>` CPU utilization.

The solution is `k8s-fix-metrics-server.sh`. It enables the metrics-server addon, detects image pull problems, patches the image when necessary, waits for rollout, and verifies metrics availability. This makes the HPA demo more reliable.

5.3 Split Backend Routing

After splitting the backend into public, admin, and monitoring deployments, routing correctness became more important. If `/api/admin/cluster` or `/api/admin` is routed to the wrong backend, the request may fail because that backend mode does not register the required routes.

The solution is to keep path-based routing consistent in both Nginx and Kubernetes Ingress:

```
/api/admin/cluster -> monitoring
/api/admin         -> admin
/api              -> public
/                 -> frontend
```

This reduces differences between NodePort access and Ingress access.

5.4 Order Consistency Under Retry or Failure

Order placement is more sensitive than simple read operations. If a client retries an order request after timeout or failover, a naive implementation may create duplicate orders and decrement stock twice.

The solution is to use both database transactions and idempotency keys. The frontend sends an `Idempotency-Key`, and the database enforces uniqueness on `(user_id, idempotency_key)`. If the same order request is retried, the backend returns the existing order instead of creating a duplicate. The transaction also ensures that order creation, item insertion, stock decrement, and cart cleanup succeed or fail together.

5.5 CloudNativePG Local Storage Permission

A major challenge in the HA database deployment was PostgreSQL data directory permission. When CloudNativePG initialized PostgreSQL on dynamically created Minikube hostPath storage, `initdb` could fail because the mounted data directory was owned by `root`, while the CloudNativePG PostgreSQL container expects to write as PostgreSQL UID/GID 26.

The final solution was to prepare local storage before creating the CloudNativePG cluster. The script `k8s-prepare-cnpg-local-storage.sh` creates node-local directories, sets the correct ownership and permissions, applies a dedicated `bookstore-cnpg-local` StorageClass, and creates static local PVs. This changes the workflow from failure recovery after a broken PVC to preventing the permission problem before database initialization starts.

5.6 Image Preflight for Recreated Minikube Profiles

Another recurring issue was that images available in host Docker were not necessarily available inside a Minikube profile. After deleting or recreating a profile, required images such as `postgres:16`, `bookstore-backend:latest`, `bookstore-frontend:latest`, and CloudNativePG images could be missing from the Minikube runtime. This caused Pods or Jobs to enter `ImagePullBackOff`.

The solution was to add an image preflight step before Kubernetes resources are created. The preflight workflow checks required host Docker images, optionally pulls missing external images, loads available images into the target Minikube profile, and fails early with clear instructions if a required image cannot be found. This makes image problems visible at build time instead of during Pod scheduling.

6 Future Work

Although the current system already demonstrates the main cloud-native features required for this project, several improvements could make it more complete and closer to a production-level system.

First, the Admin Demo should be extended with authentication and role-based authorization. In the current implementation, the admin page is designed mainly for demonstration, so future versions should restrict catalog and inventory management operations to authorized users only.

Second, the database layer can be further improved. The current project already includes a CloudNativePG-based HA mode for course-level distributed-system demonstration. A production version should further add backup and restore procedures, cross-machine or managed Kubernetes deployment, disaster recovery validation, and operational monitoring for PostgreSQL.

Third, the monitoring system can be upgraded. The current Monitoring Dashboard is lightweight and useful for showing pod status, HPA behavior, and basic metrics during the demo. Future work could add Prometheus and Grafana to provide long-term metric storage, better visualization, alerting, and more detailed performance analysis.

Finally, the deployment workflow can be made more realistic by adding CI/CD pipelines and deploying the system on a real multi-node Kubernetes cluster. This would allow automatic image building, testing, and deployment, while also providing stronger evidence of fault tolerance, service scheduling, and distributed system behavior.

7 Conclusion

This project successfully implements a cloud-native online bookstore with a complete frontend, backend, database, and Kubernetes deployment workflow. The user-facing application supports catalog browsing, search, cart management, order placement, and order history. The Admin Demo supports basic catalog and stock management. The Monitoring Dashboard provides in-app visibility into Kubernetes backend deployment, HPA, pods, metrics, and database status.

From a system architecture perspective, the project demonstrates a clear three-tier architecture and an enhanced Kubernetes backend split. The frontend is served by Nginx and proxies API traffic by path. The backend uses FastAPI and one shared image running in different modes for public, admin, and monitoring responsibilities. PostgreSQL provides persistent relational storage, while Kubernetes resources handle service discovery, configuration, health probes, routing, autoscaling, and RBAC.

From an implementation perspective, the strongest parts are the transactional order flow, idempotency-key support, path-based service separation, deployment automation, and HPA/monitoring demonstration. The project also documents realistic operational challenges such as stale Minikube images, metrics-server failures, Ingress repair, and public demo exposure.

Overall, the project satisfies the core expectations of an engineering-oriented cloud-native project. It demonstrates functionality, deployment robustness, and system-level reasoning. With additional production features such as authentication, TLS, CI/CD, real secret management, and production observability, the same architecture could be evolved into a more realistic cloud deployment.

8 Appendix A: Demo and Verification Commands

8.1 Full Distributed HA Build

```
AUTO_FIX_CNPG_PVC_PERMISSIONS=1 \  
EXPOSE_DEMO=1 \  
PUBLIC_PORT=3000 \  
NODE_PORT=30080 \  
MINIKUBE_PROFILE=bookstore-distributed \  
NODES=3 \  
CPUS=2 \  
MEMORY=4096 \  
DB_MODE=ha \  
./scripts/k8s-distributed-ha-rebuild-all.sh
```

8.2 Runtime Status Checks

```
minikube -p bookstore-distributed kubectl -- get nodes -o wide  
minikube -p bookstore-distributed kubectl -- get pods -n bookstore -o wide  
minikube -p bookstore-distributed kubectl -- get svc -n bookstore
```

8.3 Image Preflight

```
MINIKUBE_PROFILE=bookstore-distributed DB_MODE=ha \  
./scripts/k8s-preload-images.sh
```

8.4 Public Demo Exposure

```
MINIKUBE_PROFILE=bookstore-distributed \  
PUBLIC_PORT=3000 \  
NODE_PORT=30080 \  
./scripts/k8s-expose-demo.sh
```

```
MINIKUBE_PROFILE=bookstore-distributed \  
PUBLIC_PORT=3000 \  
NODE_PORT=30080 \  
./scripts/k8s-check-demo-exposure.sh
```

8.5 Functional and Consistency Tests

```
MINIKUBE_IP=$(minikube -p bookstore-distributed ip)  
  
BASE_URL=http://$MINIKUBE_IP:30080 ./scripts/test-api.sh  
BASE_URL=http://$MINIKUBE_IP:30080 ./scripts/test-admin-api.sh  
BASE_URL=http://$MINIKUBE_IP:30080 ./scripts/test-order-consistency.sh
```

8.6 HA and HPA Experiments

```
MINIKUBE_PROFILE=bookstore-distributed DB_MODE=ha \  
./scripts/k8s-distributed-ha-evidence.sh  
  
MINIKUBE_PROFILE=bookstore-distributed DB_MODE=ha \  
./scripts/k8s-postgres-ha-status.sh  
  
MINIKUBE_PROFILE=bookstore-distributed DB_MODE=ha \  
./scripts/k8s-postgres-ha-failover-test.sh  
  
MINIKUBE_PROFILE=bookstore-distributed \  
./scripts/k8s-fix-metrics-server.sh  
  
MINIKUBE_PROFILE=bookstore-distributed DURATION=240s CONCURRENCY=30 \  
./scripts/k8s-hpa-demo.sh
```